

PROJECT REPORT

STUDENT EXPENSE TRACKER

A Java-Based Personal Expense Management System

Course: Programming in Java

Submitted By:

Name:

AKSHARA KHANDELWAL

Registration Number: 25BAI10408

VIT Bhopal University

Academic Year: 2026–27

2. Introduction:-

Student Expense Tracker is a command-line Java app designed to help students record and review their everyday expenses. The project demonstrates object-oriented programming, collections, file handling, exception handling, input validation and modular programming.

3. Problem Statement:- Students often spend money on food, travel, stationery, entertainment and other activities.

Without maintaining proper records, it becomes difficult to know the total amount spent and identify high-spending categories.

The Student Expense Tracker provides a simple Java-based solution to record, view, search, delete and analyse expenses

4. Objectives

- To develop a Java-based expense management system.
- To record and manage student expenses.
- To apply OOP concepts in a practical project.
- To store expense records using file handling.
- To provide expense analysis and budget monitoring.
- To validate user input and handle errors.
- To test important expense-management operations.

5. Functional Requirements:-

Module 1 — Expense Management

- Add expense
- View all expenses
- Search expense
- Delete expense

Module 2 — Expense Analysis

- Calculate total expenses
- Calculate average expense
- Find highest expense
- Category-wise expense summary

Module 3 — Data Storage & Validation

- Save expenses to local file
- Load expenses when application starts
- Validate amount and menu input
- Handle invalid input

Module 4 — Budget Monitoring

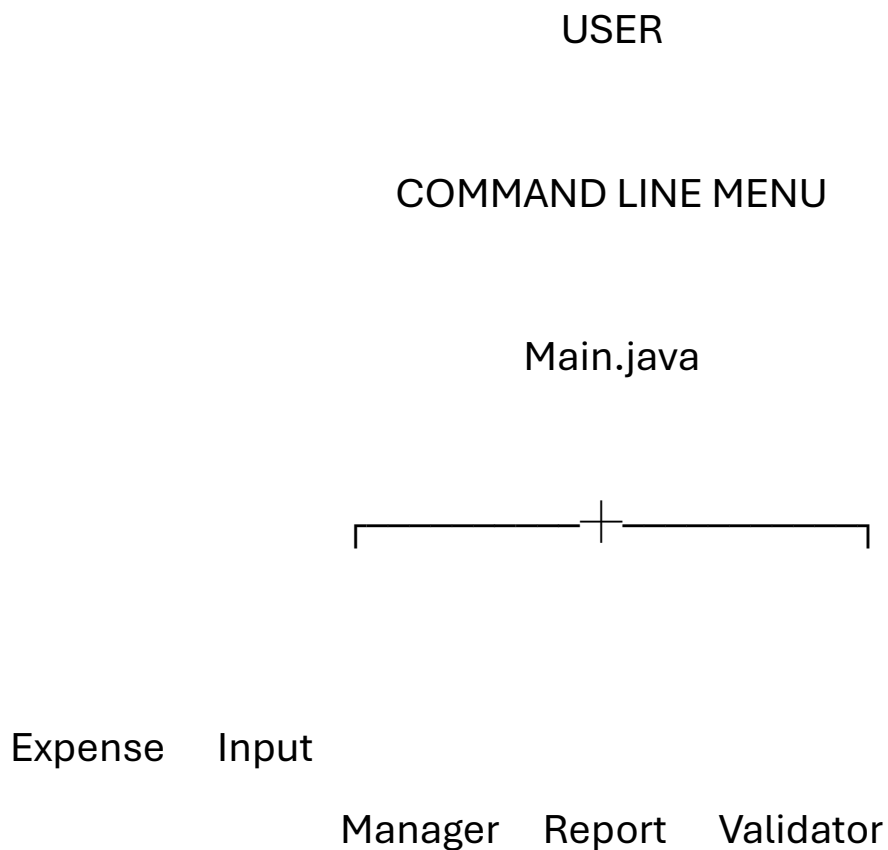
- User enters monthly budget .
- System calculates total spending .
- Calculates remaining budget.
- Shows warning if budget is exceeded .
- Shows alert if user is close to budget limits.

6. Non-Functional Requirements

Requirement	Description
Usability-	Simple command-line menu makes the application easy to use.
Performance-	Expense operations are fast for normal student-sized data.
Reliability-	Data is stored locally so records can be loaded again.

Maintainability-	Different responsibilities are divided into separate Java classes.
Error Handling-	Invalid numeric and menu inputs are handled using validation and exception handling.

7. System Architecture



Expense Analysis
Records

FileManager

expenses.txt



8. WORKFLOW DIAGRAM

START



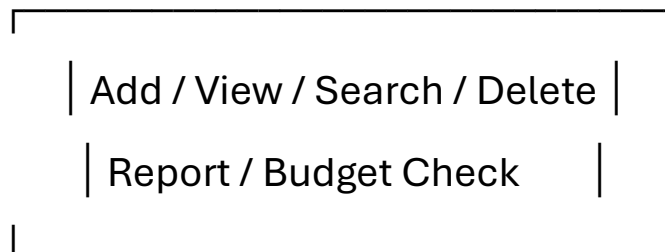
Load Saved Expenses



Display Menu



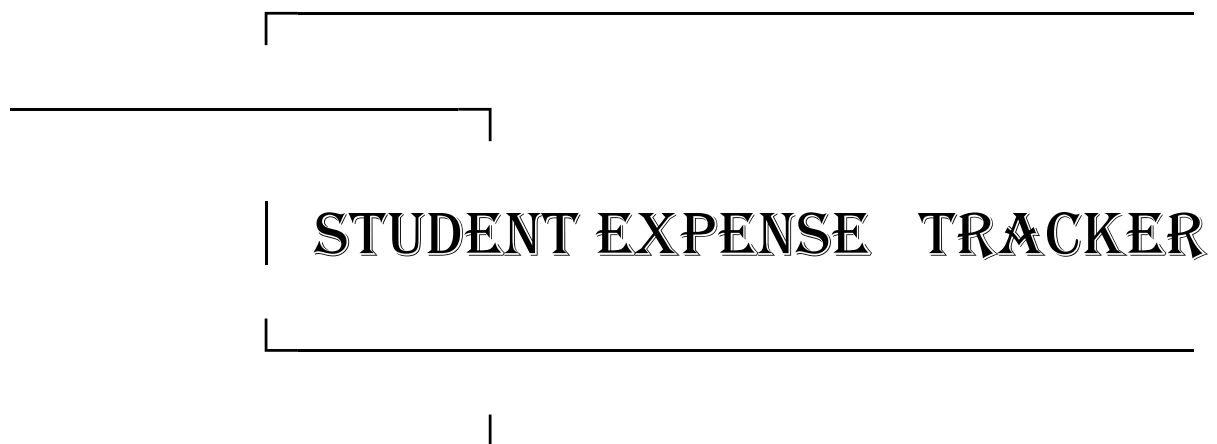
Select Operation



Return to Menu



9. USE CASE DIAGRAM



STUDENT —————→ ADD EXPENSE

STUDENT —————→ VIEW EXPENSES

STUDENT —————→ SEARCH

EXPENSE

STUDENT —————→ DELETE EXPENSE

STUDENT —————→ EXPENSE
REPORT

STUDENT —————→ BUDGET CHECK

STUDENT —————→ EXIT

10. DESIGN DECISIONS

JAVA WAS SELECTED BECAUSE THE PROJECT IS DEVELOPED FOR THE PROGRAMMING IN JAVA COURSE. ARRAYLIST IS USED BECAUSE THE NUMBER OF EXPENSES CAN CHANGE DURING PROGRAM EXECUTION. SEPARATE CLASSES WERE CREATED TO DIVIDE RESPONSIBILITIES AND MAKE THE PROGRAM EASIER TO MAINTAIN. LOCAL FILE STORAGE WAS SELECTED TO AVOID REQUIRING AN EXTERNAL DATABASE.

11. IMPLEMENTATION DETAILS:-

<i><u>Class</u></i>	<i><u>Purpose</u></i>
<i><u>Main.java</u></i>	<i><u>Controls the main program workflow</u></i>
<i><u>Expense.java</u></i>	<i><u>Represents an expense record</u></i>
<i><u>ExpenseManager.java</u></i>	<i><u>Adds, searches and deletes expenses</u></i>
<i><u>ExpenseReport.java</u></i>	<i><u>Calculates expense statistics</u></i>
<i><u>FileManager.java</u></i>	<i><u>Saves and loads expense data</u></i>

<u>InputValidator.java</u>	<u>Validates user input</u>
<u>Menu.java</u>	<u>Displays menu options</u>
<u>TestExpenseManager.java</u>	<u>Tests basic operations</u>

12.WORKING SCREENSHOTS FOR YOUR REFERENCE:-

File Edit Selection View Go ...

StudentExpenseTracker

Expense.class X

```
1 // Source code is decompiled from
2 public class Expense {
3     private int id;
4     private double amount;
5     private String category;
6     private String description;
7     private String date;
8
9     public Expense(int var1, double
10         this.id = var1;
11         this.amount = var2;
12         this.category = var4;
13         this.description = var5;
14         this.date = var6;
```

Problems Output Debug Console Terminal

```
PS C:\Users\user\Downloads\StudentExpenseTracker> javac src/*.java
src\Main.java:12: error: method loadExpenses() in class ExpenseManager
cannot be applied to given types;
    FileManager.loadExpenses(manager);
                  ^
  required: no arguments
  found:    ExpenseManager
  reason: actual and formal argument lists differ in length
src\Main.java:24: error: method readMenuChoice() in class InputValidator
cannot be applied to given types;
    int choice = InputValidator.readMenuChoice();
                              ^
  required: Scanner,int,int
  found:    Scanner,String,int,int
```

```
er.cl...
er.ja...
.class
.java
ass
va
class
.java
anag...

6. Check Monthly Budget
7. Exit
=====
Enter your choice: 6

--- Budget Check ---
Enter your monthly budget: 60000

----- Budget Status -----
Your budget: Rs. 60000.00
Amount spent: Rs. 70000.00
WARNING: You have exceeded your budget!

===== MENU =====
1. Add Expense
2. View All Expenses
3. Search Expense
4. Delete Expense
5. Expense Report
6. Check Monthly Budget
7. Exit
=====
Enter your choice: █
```

src

Expense.class

Expense.java

ExpenseManager.cl...

ExpenseManager.ja...

ExpenseReport.class

ExpenseReport.java

FileManager.class

FileManager.java

InputValidator.class

InputValidator.java

Main.java

Menu.java

TestExpenseManag...

.gitignore

README.md

statement.md

> Outline

> Timeline

> Java Projects

Problems

Output

Debug Console

Terminal

Port

```
7. Exit
=====
Enter your choice: 6

--- Budget Check ---
Enter your monthly budget: 60000

----- Budget Status -----
Your budget: Rs. 60000.00
Amount spent: Rs. 70000.00
WARNING: You have exceeded your budget!

===== MENU =====
1. Add Expense
2. View All Expenses
3. Search Expense
4. Delete Expense
5. Expense Report
6. Check Monthly Budget
7. Exit
=====
Enter your choice: 7
Thank you for using Student Expense Tracker!
PS C:\Users\user\Downloads\StudentExpenseTrac
```

```
er.cl...
er.ja...
.class
.java
ass
va
class
.java
anag...

6. Check Monthly Budget
7. Exit
=====
Enter your choice: 6

--- Budget Check ---
Enter your monthly budget: 60000

----- Budget Status -----
Your budget: Rs. 60000.00
Amount spent: Rs. 70000.00
WARNING: You have exceeded your budget!

===== MENU =====
1. Add Expense
2. View All Expenses
3. Search Expense
4. Delete Expense
5. Expense Report
6. Check Monthly Budget
7. Exit
=====
Enter your choice: █
```

⚠ 0 Java: Ready Add Comments Generate Commit Message

13.TESTING

<u>Test Case</u>	<u>Expected Result</u>	<u>Actual Result</u>
<u>Add valid expense</u>	<u>Expense added successfully</u>	<u>Passed</u>
<u>View expenses</u>	<u>Stored expenses displayed</u>	<u>Passed</u>
<u>Search expense</u>	<u>Matching record displayed</u>	<u>Passed</u>
<u>Delete existing ID</u>	<u>Expense deleted</u>	<u>Passed</u>

<u>Delete invalid ID</u>	<u>Error message displayed</u>	<u>Passed</u>
<u>Invalid amount</u>	<u>User asked for valid amount</u>	<u>Passed</u>
<u>Budget check</u>	<u>Budget status displayed</u>	<u>Passed</u>
<u>Automated test</u>	<u>All basic tests pass</u>	<u>Passed</u>

14. Challenges Faced

- Understanding how ArrayList stores multiple expense records.
- Handling file input and output.
- Managing invalid user input.
- Dividing the program into multiple classes.
- Debugging compilation errors.
- Testing different menu operations.

Report instructions specifically recommend completing this from your actual experience.

15.LEARNING

Through this project, I learned how Java classes and objects can be used to solve a real-world problem. I also learned about ArrayList, encapsulation, file handling,

**exception handling, input validation,
modular programming and testing.**

16. Future Enhancements

- Graphical User Interface
- Database integration
- CSV/PDF report export
- Date-based monthly reports
- Login/password protection
- Mobile/web version
- Expense charts and visualizations

17 REFERENCE

1. Oracle Java Documentation

2. Java course learning material

3. VITyarthi Build Your Own Project guidelines